

Miniprojeto Guiado — Livraria

Introdução à Orientação a Objetos

Aula 09: números, strings, datas e listas

Instruções

- Trabalhem em **duplas**. A cada etapa, troquem quem está digitando.
- Baixem no Moodle o arquivo `aula09_projeto_partida.zip`, descompactem e abram a pasta `src` na IDE. O projeto é o da aula 06: `Autor` e `CarrinhoDeCompras` em `br.edu.ifsp.livraria`; `Livro` em `br.edu.ifsp.livraria.produtos`; `CadastroDeLivros`, com o `main`, em `br.edu.ifsp.livraria.testes`.
- Rodem o `main` de `CadastroDeLivros`. A saída deve ser:

```
Livros cadastrados: 2
Total do carrinho: 148.9
```

- Cada etapa tem **requisitos**, um **teste** que vocês acrescentam ao **final** do `main` e a **saída esperada**. Só passem para a etapa seguinte quando a saída bater.
- Podem consultar a trilha da aula 08 e os livros da disciplina.
- Sigam as convenções de nomes do Java e escrevam comentários em português, só quando necessário.
- Com o `Autor`, não é preciso mexer: ele já está pronto.

Observações: as três armadilhas

Elas aparecem em todas as etapas. Releiam antes de começar e voltem aqui quando a saída não bater.

1. `String` e `LocalDate` são **imutáveis**: **guarde o retorno**. Métodos como `trim()` e `toUpperCase()` devolvem um **novo** objeto; o original não muda.

```
titulo.trim().toUpperCase();           // nao muda nada
titulo = titulo.trim().toUpperCase();   // agora sim
```

2. **Texto se compara com equals, nunca com ==**. O `==` compara referências (se são o mesmo objeto); o `equals` compara o texto. Para ignorar maiúsculas e minúsculas, `equalsIgnoreCase`.

```
String a = "livro";
String b = new String("livro");
System.out.println(a == b);           // false
System.out.println(a.equals(b));      // true
```

3. **Texto que vira número pode falhar.** `Double.parseDouble("abc")` lança `NumberFormatException`: use `try/catch`. E, nas listas, o tipo genérico precisa ser uma classe: `List<Integer>`, nunca `List<int>`.

```
try {
    double valor = Double.parseDouble(texto);
} catch (NumberFormatException e) {
    System.out.println("Nao e um numero");
}
```

Etapa 1 — Livro com texto limpo

Objetivo: garantir que título, ISBN e valor cheguem à classe já validados.

1. Faça o `setNome` **padronizar** o título: sem espaços no início e no fim, e em letras maiúsculas.
2. Crie `public static boolean isbnValido(String isbn)`: o ISBN é válido se, **depois de retirar os traços**, tiver exatamente 13 caracteres e **todos forem dígitos**. Percorra o texto com `charAt`.
3. **Altere** o `setIsbn` para `public boolean setIsbn(String isbn)`: se o ISBN for válido, grava **sem os traços** e devolve `true`; senão, não altera nada e devolve `false`. Ele deve ser o **único** caminho para gravar o ISBN.
4. Crie, por **sobrecarga** do `setValor(double)`, o método `public boolean setValor(String texto)`: converte o texto em `double`. Se o texto não for um número, ou se o número for negativo, não altera o valor e devolve `false`; senão, grava e devolve `true`. Use `try/catch`.

Teste (acrescente ao final do `main`):

```
System.out.println("--- Etapa 1 ---");
Livro livro = new Livro(new Autor("Herbert Schildt"));
livro.setNome("    java para iniciantes    ");
System.out.println "[" + livro.getNome() + " ]";
System.out.println(livro.setIsbn("978-85-66250-46-6"));
System.out.println(livro.setIsbn("978-85-662"));
System.out.println(livro.setIsbn("978-85-6625A-46-6"));
System.out.println(livro.getIsbn());
System.out.println(livro.setValor("59.90"));
System.out.println(livro.setValor("59,90"));
System.out.println(livro.setValor("abc"));
System.out.println(livro.getValor());
```

Saída esperada:

```

--- Etapa 1 ---
[JAVA PARA INICIANTE]
true
false
false
9788566250466
true
false
false
59.9

```

Atenção: "978-85-6625A-46-6" tem 13 caracteres, mas um deles não é dígito. E lembre que os métodos de `String` **não alteram** o texto original: guardem o retorno.

Etapa 2 — Livro com data

Objetivo: registrar a data de publicação e mostrar os dados no formato brasileiro.

1. Acrescente o atributo `LocalDate dataPublicacao`, com `set` e `get`.
2. Crie `public int anosDesdeAPublicacao(LocalDate referencia)`, que devolve os anos **completos** entre a publicação e a data de referência.
3. Crie, por **sobrecarga**, `public int anosDesdeAPublicacao()` sem parâmetro, que usa a data de hoje. Reaproveite o método do item anterior.
4. Altere `mostrarDetalhes()` para imprimir o valor como R\$ 59,90 e a data como 15/01/2014 (linha Publicado em: ...). O formatador da data deve ser uma **constante** da classe. Se a data ainda não foi informada, essa linha não é impressa.

Teste (importe `java.time.LocalDate` no `main`):

```

System.out.println("--- Etapa 2 ---");
livro.setDataPublicacao(LocalDate.of(2014, 1, 15));
System.out.println(livro.anosDesdeAPublicacao(LocalDate.of(2026, 10, 1))
);
System.out.println(livro.anosDesdeAPublicacao(LocalDate.of(2024, 1, 14))
);
livro.mostrarDetalhes();

```

Saída esperada:

```

--- Etapa 2 ---
12
9
Nome: JAVA PARA INICIANTE
ISBN: 9788566250466
Valor: R$ 59,90
Publicado em: 15/01/2014
Autor: Herbert Schildt
--

```

Atenção: de 15/01/2014 a 14/01/2024 falta **um dia** para 10 anos completos. Em dd/MM/yyyy, MM é mês e mm é minuto. Se o computador estiver configurado em inglês, o valor pode sair R\$ 59.90: o separador decimal segue o idioma.

Etapa 3 — CarrinhoDeCompras com List

Objetivo: trocar o array de 10 posições por uma lista que cresce sozinha.

1. Troque o array por um atributo `List<Livro> livros`, criado como `ArrayList`. Apague o contador e o limite de 10 livros.
2. Mantenha `adiciona(Livro)` e `remove(int posicao)`. Crie, por **sobrecarga**, `remove(Livro livro)`, que remove o **próprio** livro.
3. Crie `public int getQuantidade()`.
4. `getTotal()` continua somando o valor dos livros. Como a lista não tem buracos, o teste de `null` deixa de ser necessário.
5. `getLivros()` passa a devolver `List<Livro>`.

O código que já estava no `main` **não deve precisar de alteração**.

Teste (acrescente ao final do `main`):

```
System.out.println("--- Etapa 3 ---");
carrinho.adiciona(livro);
System.out.println(carrinho.getQuantidade());
System.out.println(carrinho.getTotal());
carrinho.remove(0);
System.out.println(carrinho.getQuantidade());
System.out.println(carrinho.getTotal());
carrinho.remove(livro);
System.out.println(carrinho.getQuantidade());
for (Livro item : carrinho.getLivros()) {
    System.out.println(item.getNome());
}
```

Saída esperada:

```
--- Etapa 3 ---
3
208.8
2
148.9
1
USE A CABECA! JAVA
```

Atenção: depois de `remove(0)` a lista se reorganiza: não sobra buraco. O `Livro` usado no teste é o da Etapa 1 (R\$ 59,90).

Etapa 4 — Catalogo com buscas

Objetivo: guardar e procurar livros por título, trecho, data e categoria.

Crie a classe `Catalogo` no pacote `br.edu.ifsp.livraria`, com um atributo `List<Livro>` e outro `List<String>` para as categorias:

1. `void cadastra(Livro livro)` e `int getTotalDeLivros()`.
2. `Livro buscaPorTitulo(String titulo)`: ignora diferença entre maiúsculas e minúsculas e espaços no início e no fim; devolve `null` se não encontrar.

3. `List<Livro> buscaPorTrecho(String trecho)`: devolve os livros cujo título **con-**
tém o trecho, também ignorando maiúsculas e minúsculas.
4. `List<Livro> publicadosAntes(LocalDate limite)`: devolve os livros publicados
antes da data. Livros sem data são ignorados.
5. `boolean adicionaCategoria(String categoria)`: guarda a categoria **padroni-**
zada (sem espaços no início e no fim e em minúsculas). Se ela já existir, não guarda
e devolve `false`.
6. `List<String> getCategorias()`.

Teste (acrescente ao final do `main` e importe `Catalogo`):

```
System.out.println("--- Etapa 4 ---");
desbravando.setDataPublicacao(LocalDate.of(2014, 1, 15));
cabeca.setDataPublicacao(LocalDate.of(2005, 5, 20));
Catalogo catalogo = new Catalogo();
catalogo.cadastra(desbravando);
catalogo.cadastra(cabeca);
catalogo.cadastra(livro);
System.out.println(catalogo.getTotalDeLivros());
System.out.println(catalogo.buscaPorTitulo(" desbravando java ").
    getNome());
System.out.println(catalogo.buscaPorTitulo("Java Avancado"));
System.out.println(catalogo.buscaPorTrecho("java").size());
System.out.println(catalogo.publicadosAntes(LocalDate.of(2010, 1, 1)).
    size());
System.out.println(catalogo.adicionaCategoria(" Programacao "));
System.out.println(catalogo.adicionaCategoria("programacao"));
System.out.println(catalogo.adicionaCategoria("Java"));
System.out.println(catalogo.getCategorias());
```

Saída esperada:

```
--- Etapa 4 ---
3
DESBRAVANDO JAVA
null
3
1
true
false
true
[programacao, java]
```

Atenção: compare `String` com `equals` ou `equalsIgnoreCase`, nunca com `==`.

Desafio bônus (para casa)

Em `Catalogo`, crie `public int aplicaPromocaoParaAntigos(int anos, double porcentagem, LocalDate referencia)`: aplica o desconto nos livros publicados há **pelo menos** `anos` (na data de referência) e devolve **quantos livros receberam o desconto**. Livros sem data ficam de fora. Reaproveite `anosDesdeAPublicacao` e `aplicaDescontoDe`, e lembre do limite `DESCONTO_MAXIMO` da classe `Livro`.

Teste:

```
System.out.println("--- Bonus ---");
System.out.println(catalogo.aplicaPromocaoParaAntigos(10, 0.2, LocalDate
    .of(2026, 10, 1)));
System.out.println(catalogo.aplicaPromocaoParaAntigos(20, 0.5, LocalDate
    .of(2026, 10, 1)));
System.out.println(desbravando.getValor());
```

Saída esperada:

```
--- Bonus ---
3
Desconto acima do permitido
0
47.92
```

Entrega

Compactem a pasta `src` do projeto em um arquivo `.zip` com o nome

`livraria_Sobrenome1_Sobrenome2.zip`

e enviem pela tarefa do Moodle, dentro do prazo indicado. O projeto precisa **compilar** e o `main` deve imprimir a saída de todas as etapas concluídas.

Lista de conferência

- ☐ O projeto compila sem erros.
- ☐ A saída de cada etapa bate com a saída esperada.
- ☐ Nenhum `String` é comparado com `==`.
- ☐ O `setIsbn` valida o ISBN e não há outro caminho para gravar um ISBN inválido.
- ☐ O `main` original continuou funcionando depois da Etapa 3.
- ☐ Os nomes seguem as convenções do Java.